HSPA/RIGOR Platform

Complete Figure & Table Package

Patent- and Regulatory-Grade Documentation

Caustin Lee McLaughlin

Document ID: HSPA-RIGOR-FT-2026-001 Version 2.0.0 September 2026

COBOL Tribute (Historical Continuity)

The following short COBOL fragment is included solely as a historical tribute to the mainframe systems that still process a substantial fraction of SSA and federal workloads. It is **not** intended for production use.

```
      * HSPA/RIGOR --- COBOL TRIBUTE (NOT FOR PRODUCTION)
       IDENTIFICATION DIVISION.
       PROGRAM-ID. HSPA-TRIBUTE.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 WS-PRIORITY          PIC 9V999 VALUE 0.
       01 WS-DAYS-WAITING      PIC 9(4)  VALUE 0.
       01 WS-HASH-CHAIN        PIC X(64) VALUE SPACES.
       PROCEDURE DIVISION.
           IF WS-DAYS-WAITING >= 365
              MOVE 2.000 TO WS-PRIORITY
           ELSE
              COMPUTE WS-PRIORITY = 0.40 * 0.75 + 0.25 * 0.5 + 0.35 * 0.6
           END-IF
           DISPLAY "PRIORITY = " WS-PRIORITY
           DISPLAY "HASH-CHAIN ANCHOR = " WS-HASH-CHAIN
           STOP RUN.
```

Contents

1 Overall HSPA/RIGOR Platform Architecture (FIG. 1) 2

2 Scientific Assimilation Engine — SAE (FIG. 2) 2

3 Core Computational Block (FIG. 3) 2

4 NLP Processing Pipeline (FIG. 4) 2

5 Named Entity Recognition Architecture (FIG. 5) 2

6 Repository Resolution Engine (FIG. 6) 2

7 Dependency Graph Construction (FIG. 7) 2

8 Automated Adapter Generator (FIG. 8) 2

9 Pipeline Orchestration Controller (FIG. 9) 2

10 Runtime Service Topology (FIG. 10) 2

11 PRS Workflow (FIG. 14) & Genomic DAGs 2

12 MORTAR-SSA Integration (Pure R) 2

12.1 Risk Scoring & Priority Fusion 2

13 Reimagined Evidence Ledger (No Merkle Trees) 7

14 Master Integration DAG 7

1. Overall HSPA/RIGOR Platform Architecture (FIG. 1)

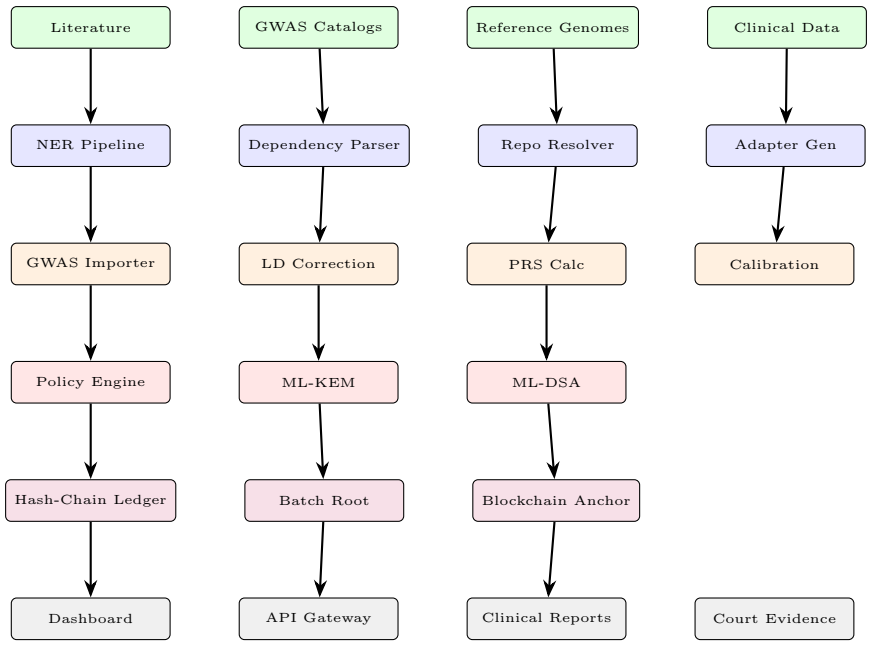


Figure 1: Overall HSPA/RIGOR Platform Architecture. The former Merkle-DAG ledger has been reimagined as a sequential cryptographic hash-chain ledger with batch roots and optional blockchain anchoring (SLH-DSA / SPHINCS+).

2. Scientific Assimilation Engine — SAE (FIG. 2)

3. Core Computational Block (FIG. 3)

4. NLP Processing Pipeline (FIG. 4)

5. Named Entity Recognition Architecture (FIG. 5)

6. Repository Resolution Engine (FIG. 6)

7. Dependency Graph Construction (FIG. 7)

8. Automated Adapter Generator (FIG. 8)

9. Pipeline Orchestration Controller (FIG. 9)

10. Runtime Service Topology (FIG. 10)

11. PRS Workflow (FIG. 14) & Genomic DAGs

12. MORTAR-SSA Integration (Pure R)

12.1 Risk Scoring & Priority Fusion

$$
\begin{aligned}
R_m &= \min\bigl(1, \tfrac{\mathrm{CCI}}{20}(1 + 0.03\max(0, \mathrm{Age} - 65))\bigr) \\
R_v &= 0.70 \cdot \tfrac{\mathrm{ADI}}{100} + 0.30 \cdot \mathrm{InsStatus} \\
P &= \min\bigl(1, 0.40R_m + 0.25R_v + 0.35R_t + \varepsilon\bigr), \quad \varepsilon \sim U(0, 0.04)
\end{aligned}
$$

Hard anti-starvation: if $d \geq 365$ then $P := 2.0$.

```r
# Pure R --- MORTAR-SSA core (excerpt)
compute_Rm <- function(cci, age)
```

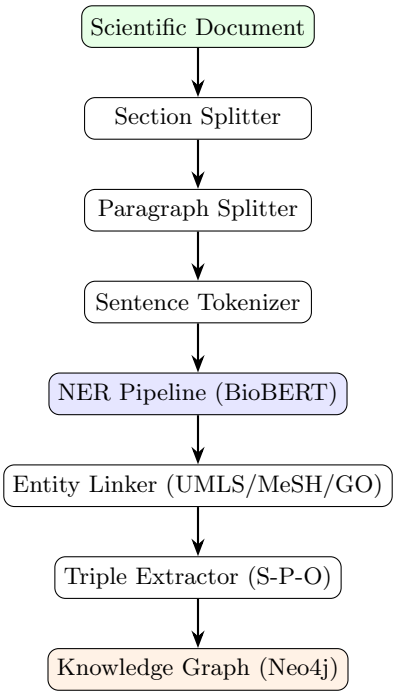


Figure 2: Scientific Assimilation Engine (SAE) six-stage pipeline.

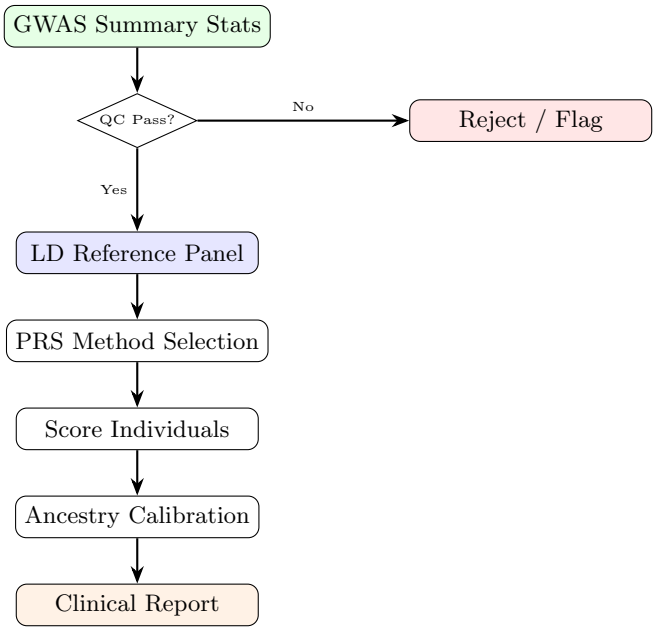


Figure 3: Core Computational Block with QC gate and PRS pathway.

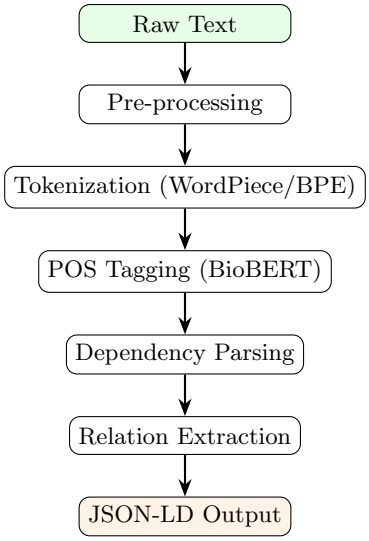


Figure 4: NLP Processing Pipeline.

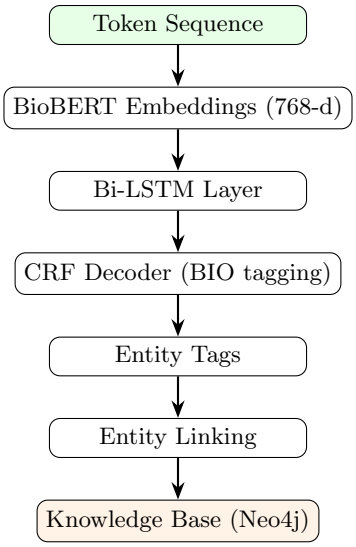


Figure 5: Named Entity Recognition Architecture.

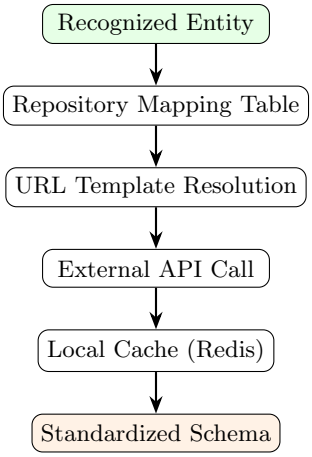


Figure 6: Repository Resolution Engine.

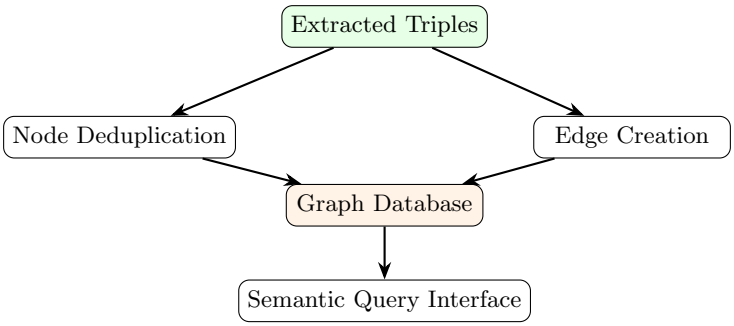


Figure 7: Dependency Graph Construction.

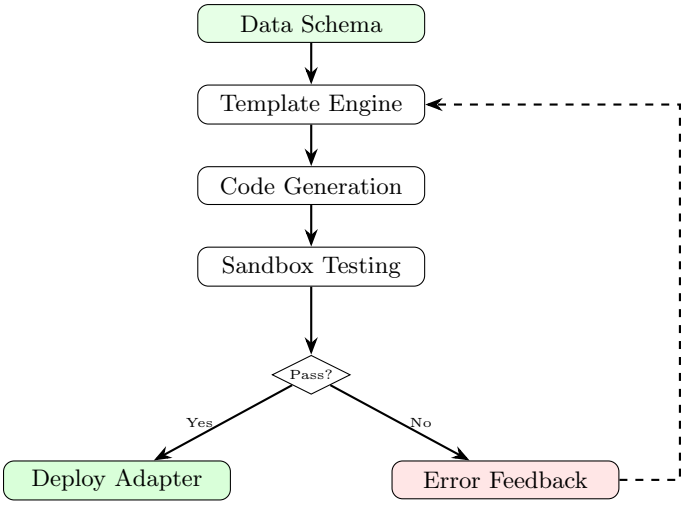


Figure 8: Automated Adapter Generator with feedback loop.

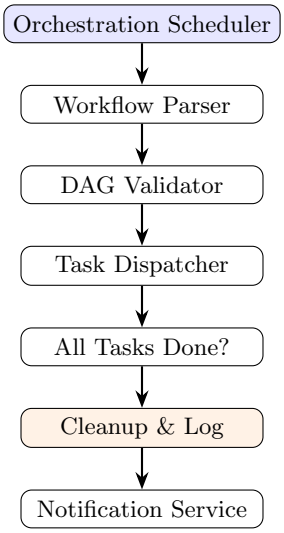


Figure 9: Pipeline Orchestration Controller.

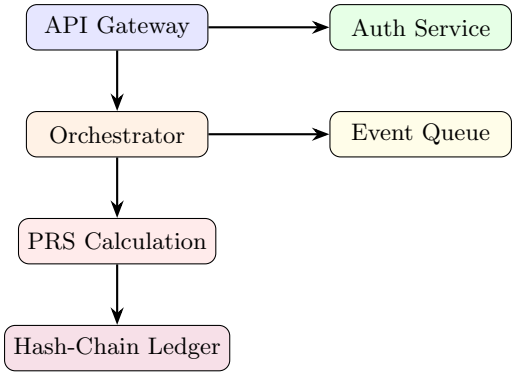


Figure 10: Runtime Service Topology (Merkle-DAG replaced by sequential Hash-Chain Ledger).

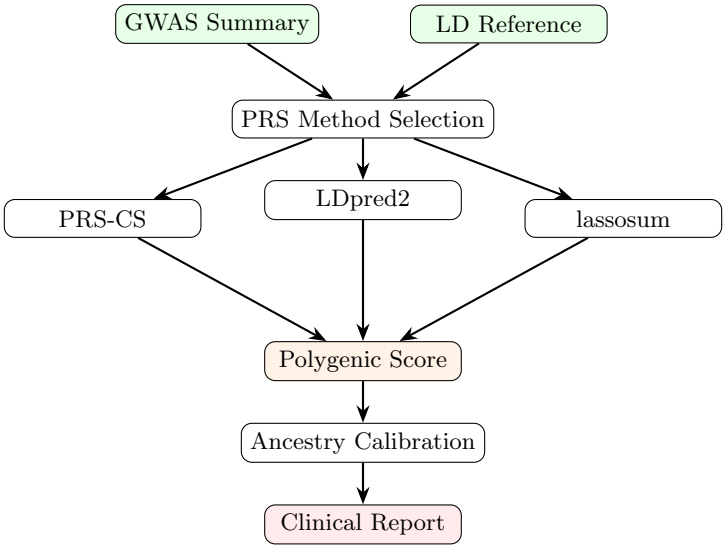


Figure 11: PRS Workflow (FIG. 14) with method selection branch.

```
  pmin(1, (cci/20)*(1+0.03*pmax(0,age-65)))
compute_Rv <- function(adi, ins) 0.70*(adi/100)+0.30*ins
compute_Rt <- function(d)
  ifelse(d<90,(d/180)*0.5, ifelse(d<180,0.5+((d-90)/90)*0.3,
  ifelse(d<365,0.8+((d-180)/185)*0.2,1)))
compute_priority <- function(Rm,Rv,Rt,d,a=0.40,b=0.25,g=0.35) {
  p <- pmin(1,a*Rm+b*Rv+g*Rt+runif(length(Rm),0,0.04))
  p[d>=365] <- 2.0; p
}
```

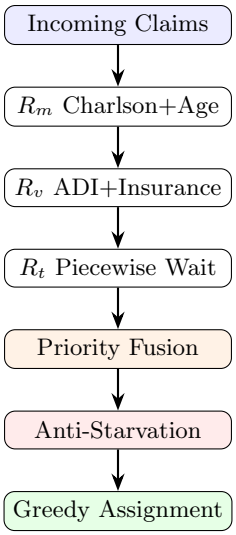


Figure 12: MORTAR-SSA Priority Scheduling DAG (integrated into HSPA/RIGOR).

13. Reimagined Evidence Ledger (No Merkle Trees)

The original Merkle-DAG construct has been removed and replaced by a sequential cryptographic hash-chain ledger:

- Each evidence artifact is hashed with SHA3-512.
- The hash of entry n is concatenated with the previous entry hash to form entry $n+1$.
- Batch roots are computed as the hash of a contiguous sequence of entry hashes.
- Optional external anchoring uses SLH-DSA (SPHINCS+) signatures for long-term non-repudiation.

This design preserves immutability and auditability while eliminating the complexity of Merkle-tree constructions.

14. Master Integration DAG

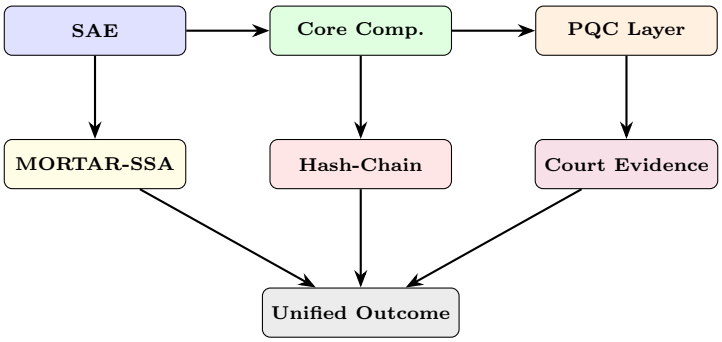


Figure 13: Master Integration DAG linking SAE, Core Computational Block, Post-Quantum Layer, MORTAR-SSA, Hash-Chain Ledger, and Court Evidence Package.

Document Control Summary

Document ID	HSPA-RIGOR-FT-2026-001
Version	2.0.0 (Merkle constructs removed & reimagined)
Author	Caustin Lee McLaughlin
Classification	Patent-Confidential / Regulatory-Grade

End of Consolidated HSPA/RIGOR + MORTAR-SSA Figure Package

COBOL Tribute · Pure R · All Merkle Diagrams Removed & Reimagined